B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019

Bio Inspired Algorithms AAT on

Air Quality Index Prediction Using Random Forest Optimized with Particle Swarm Optimization (PSO)

in fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

in

Computer Science & Engineering (Data Science)

by

Hemanth Raj MV (1BM23CD022)

Kushal KV (1BM23CD030)

Rohan R (1BM23CD051)

Shakthi Jashwanth C H (1BM23CD072)

Under the Guidance of

Dr. Kalyan N

Assistant Professor

Department of CSE (Data Science),

B.M.S. College of Engineering

AY 2026-2027

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019

Department of Computer Science and Engineering

(Data Science)

CERTIFICATE

This is to certify that the project entitled **"Air Quality Index Prediction Using Random Forest Optimized with Particle Swarm Optimization (PSO)"** is a bona fide work carried out by the following students for the course Bio Inspired Algorithms (Course Code: **23DS7BSBIO**). It is certified that all corrections and suggestions indicated during the internal assessments have been incorporated in the final report submitted to the departmental library. The project report has been approved as it meets the academic requirements prescribed for the Bachelor of Engineering Degree in respect of project work.

Candidate details

SL. NO.	Student Name	USN	Student's Signature
1	Hemanth Raj MV	1BM23CD022	
2	Kushal KV	1BM23CD030	
3	Rohan R	1BM23CD051	
4	Shakthi Jashwanth C H	1BM23CD072	

Signature of the Guide

Dr. Kalyan N
Assistant Professor

Signature of the HOD

Dr. Shyamala G
Professor & HOD

Examiners

Name of Examiner **Signature of the examiner**

1. _____

2. _____

Acknowledgements

We would take this opportunity to express our heart-felt gratitude to **Dr.B. S. Ragini Narayan**, *Chairperson*, Donor Trustee, Member Secretary & Chairperson, BMSET. **Dr. P. Dayananda Pai**, *Member Life Trustee*, BMSET and **Dr. Bheemsha Arya**, *Principal*, B.M.S. College of Engineering for providing the necessary infrastructure to complete this Alternative Assessment.

We wish to express our deepest gratitude to **Dr. Shyamala G**, Head of the Department, Computer Science and Engineering(Data Science) and the Project Guide **Dr. Kalyan N** for their constant support.

A warm thanks to all the faculty of the Department of Computer Science and Engineering(Data Science), who have helped us with their views and encouraging ideas.

Contents

Acknowledgements **2**

0.1 Introduction . . . 1
0.1.1 Background and Motivation . . . 1
0.1.2 Problem Statement . . . 1
0.1.3 Research Objectives . . . 2
0.1.4 Contributions . . . 2
0.1.5 Report Organization . . . 3
0.2 Literature Review . . . 3
0.2.1 Air Quality Forecasting Techniques . . . 3
0.2.2 Machine Learning for Air Quality Prediction . . . 3
0.2.3 Hyperparameter Optimization Methods . . . 3
0.2.4 Particle Swarm Optimization in Applications . . . 4
0.2.5 Research Gap . . . 4
0.3 Theoretical Background . . . 4
0.3.1 Time Series Forecasting . . . 4
0.3.2 National Air Quality Index . . . 5
0.3.3 Decision Trees . . . 5
0.3.4 Random Forest . . . 5
0.3.5 Particle Swarm Optimization Fundamentals . . . 6
0.3.6 Evaluation Metrics . . . 6
0.4 Nature-Inspired Algorithms: Detailed Study . . . 7
0.4.1 Overview of Nature-Inspired Optimization . . . 7
0.4.2 Particle Swarm Optimization: In-Depth Analysis . . . 7
0.4.3 Genetic Algorithm . . . 10
0.4.4 Ant Colony Optimization . . . 10
0.4.5 Simulated Annealing . . . 10
0.4.6 Differential Evolution . . . 10
0.4.7 Why PSO for Random Forest Optimization? . . . 11
0.4.8 Hybrid Approaches . . . 11
0.5 Methodology and Implementation . . . 11
0.5.1 System Architecture . . . 11
0.5.2 Data Collection . . . 12
0.5.3 Data Preprocessing . . . 13
0.5.4 Feature Engineering . . . 13
0.5.5 Random Forest Model Architecture . . . 14

0.5.6 PSO Implementation 14
0.5.7 Training Procedure 16
0.5.8 Implementation Details 17
0.6 Experimental Setup and Test Functions 18
0.6.1 Experimental Configuration 18
0.6.2 Baseline Models 18
0.6.3 Test Functions and Benchmarks 19
0.6.4 Cross-Validation Strategy 20
0.6.5 Sensitivity Analysis 21
0.6.6 Statistical Significance Testing 21
0.6.7 Computational Efficiency Metrics 22
0.7 Results and Discussion 22
0.7.1 PSO Convergence Analysis 22
0.7.2 Optimal Configuration 23
0.7.3 Feature Selection Results 24
0.7.4 Prediction Performance 26
0.7.5 Temporal Performance Analysis 27
0.7.6 Comparison with Baseline Models 28
0.7.7 Hyperparameter Importance Analysis 30
0.7.8 Feature Importance Results 31
0.7.9 Error Analysis 31
0.7.10 Computational Performance 33
0.7.11 Robustness Analysis 33
0.7.12 Discussion 34
0.8 Conclusion 36
0.9 Future Work 37
0.9.1 Methodological Enhancements 37
References 38

List of Figures

1 System architecture of the PSO-optimized Random Forest pipeline 12
2 PSO convergence on the three benchmark functions (5 runs each, log scale) . . . 20
3 PSO convergence on walk-forward validation RMSE (5 runs) 23
4 Swarm mean fitness, diversity and inertia weight in the selected run 23
5 Particle trajectories through each hyperparameter in the selected run 24
6 Share of PSO runs that kept each feature (left) and the number of features used by the swarm in the selected run (right) 25
7 Test metrics of persistence, the default forest and the final PSO model 26
8 Observed and forecast AQI over the test period, with the 2024–25 winter in detail 27
9 Test RMSE of every model 28
10 Best validation RMSE found by PSO and random search for the same evaluation budget 29
11 Feature importance of the final PSO model 31
12 Residual distribution and mean absolute error by month 32
13 CPCB category confusion matrices on the test period 33
14 Learning curves of the default and final PSO Random Forest 34
15 Model performance before optimization (left, default RF) and after PSO optimization (right) 36

Abstract

Air pollution is a severe public health problem in Delhi, where the official Air Quality Index (AQI) exceeded 300 (*Very Poor* or worse) on 22.7% of days between 2015 and 2026. This project forecasts Delhi's next-day AQI using a Random Forest regressor optimized by Particle Swarm Optimization (PSO), a nature-inspired metaheuristic based on the social behaviour of bird flocks. The dataset combines 4,170 days of official Central Pollution Control Board (CPCB) AQI bulletins (May 2015 to October 2026) with daily ERA5 weather reanalysis, giving 32 leakage-free features of AQI history, meteorology and season. PSO was implemented from first principles with linearly decreasing inertia, velocity clamping and reflecting boundaries. Each particle encodes six Random Forest hyperparameters and 32 threshold-based feature switches, so the swarm tunes the model and selects its inputs at the same time, scored by walk-forward validation error. Training used 2015 to 2023, and the 1,026 days from January 2024 to October 2026 were held out for testing. The joint search reduced validation RMSE by 4.3% (41.52 to 39.73), twice the gain of a hyperparameter-only search, kept 14 of 32 features, and outperformed random search with the same 312-evaluation budget. On the test period, the selected model reached an RMSE of 33.77, an MAE of 25.15 and an R^2 of 0.882, improving on a naive persistence forecast by 16.0%; averaged over five runs, joint PSO gave a test RMSE of 33.47, 1.0% below the default Random Forest. The selected model's gain over the default forest was small (0.25%) and not statistically significant ($p = 0.76$), and it was slightly worse than the hyperparameter-only model (RMSE 33.57), revealing that the larger search partly overfitted its validation folds. An untuned ridge regression performed comparably (RMSE 33.45), showing that informative features matter more than tuning for this problem. The work provides a reproducible, tested framework and an honest assessment of what nature-inspired optimization contributes to air quality forecasting.

0.1 Introduction

0.1.1 Background and Motivation

Air pollution is one of the most serious public health problems in India, and Delhi is its most visible example. The Central Pollution Control Board (CPCB) reports a daily National Air Quality Index (AQI) for each monitored city, on a scale from 0 to 500 divided into six categories, from *Good* to *Severe* [17]. Between May 2015 and October 2026, Delhi's official daily AQI averaged 211 (*Poor*) and exceeded 300 (*Very Poor* or worse) on 945 of 4,170 reported days (22.7%). Monthly means range from 99 in August to 351 in November.

A forecast of tomorrow's AQI is valuable because it gives time to act before the pollution arrives:

- Hospitals and individuals with respiratory illness can prepare in advance.
- Schools and outdoor workers can adjust their schedules.
- Authorities can trigger graded emergency measures one day earlier.

Delhi's air quality is hard to forecast because it depends on several interacting factors:

- Strong seasonality, with severe smog from October to February
- Meteorology: low temperature, high pressure and weak wind trap pollutants near the ground [16]
- Episodic sources such as crop-residue burning and festival fireworks
- Day-to-day persistence combined with sudden jumps and drops

Machine learning models can learn these non-linear relationships from historical data. Their accuracy, however, depends heavily on hyperparameters, which are usually chosen by hand.

0.1.2 Problem Statement

A Random Forest [4] is well suited to tabular environmental data, but its performance depends on several interacting hyperparameters: the number of trees, tree depth, leaf size, split size, feature sampling and sample fraction. Manual tuning of these hyperparameters is:

- Time-consuming, because each trial requires training several forests
- Prone to human bias and limited exploration of the search space
- Inefficient, since grid search grows exponentially with the number of hyperparameters

This project addresses the problem of automatically tuning a Random Forest for next-day AQI forecasting in Delhi using Particle Swarm Optimization (PSO), a nature-inspired metaheuristic based on the collective behaviour of bird flocks [1].

Formally, given the AQI history and weather up to the end of day t, the task is to predict the officially reported AQI of day $t+1$. Predicting the same day's AQI from that day's pollutant readings is avoided on purpose: AQI is computed directly from those readings, so such a model would only relearn the CPCB formula.

0.1.3 Research Objectives

The primary objectives of this project are:

1. Build a leakage-free forecasting dataset from official CPCB AQI bulletins and ERA5 weather reanalysis for Delhi, 2015 to 2026
2. Establish unoptimized baselines: persistence and a default Random Forest
3. Implement PSO from first principles, including velocity update, inertia scheduling, velocity clamping and boundary handling
4. Use PSO to tune six Random Forest hyperparameters and select the input features at the same time, against walk-forward validation error
5. Measure what feature selection adds by comparing against a hyperparameter-only PSO (ablation)
6. Evaluate the tuned model with regression, category and alert metrics, and test whether the improvement is statistically significant
7. Compare PSO against random search with the same evaluation budget, and against traditional regression models

0.1.4 Contributions

This work makes the following contributions:

- A reproducible end-to-end pipeline for PSO-optimized Random Forest AQI forecasting
- A dataset combining 11.5 years of official CPCB AQI bulletins for Delhi with daily ERA5 weather
- A from-scratch PSO implementation, verified on standard benchmark functions
- A joint PSO search over hyperparameters and features (38 dimensions), with an ablation over the hyperparameters alone that isolates what feature selection adds
- An honest evaluation that includes a random-search control, traditional-model comparison, block-bootstrap confidence intervals and paired significance tests
- Open, tested and documented source code

0.1.5 Report Organization

The remainder of this report is organized as follows. Section 0.2 reviews related literature. Section 0.3 gives the theoretical background on time series forecasting, Random Forests, the AQI and evaluation metrics. Section 0.4 presents a detailed study of nature-inspired algorithms. Section 0.5 describes the methodology and implementation. Section 0.6 details the experimental setup and test functions. Section 0.7 presents results and discussion, and Sections 0.8 and 0.9 give the conclusion and future work.

0.2 Literature Review

0.2.1 Air Quality Forecasting Techniques

Early air quality forecasting relied on statistical time series methods. The ARIMA family of Box and Jenkins [8] models a series as a linear function of its own past values and past errors. These models work well for linear, stationary behaviour but struggle with the non-linear, weather-driven dynamics of urban pollution.

Physical chemistry-transport models simulate emissions, transport and chemistry directly. They are physically grounded but computationally expensive, and they depend on accurate emission inventories, which are uncertain for Indian cities.

0.2.2 Machine Learning for Air Quality Prediction

Data-driven models learn the mapping from recent pollution and weather to future pollution directly from data. Tree ensembles such as Random Forests [4] handle non-linear effects, interactions and correlated inputs without feature scaling, and they work well on the few thousand daily samples typical of city-level records. Recurrent neural networks such as LSTMs [9] can model long sequences but need more data and careful tuning.

Meteorology is a key driver of pollution episodes in Delhi. Winter inversions, low wind speed and low mixing heights trap emissions near the surface [16], so weather variables are essential inputs to a forecasting model.

0.2.3 Hyperparameter Optimization Methods

Hyperparameter optimization has evolved from manual and grid search to more efficient methods. Bergstra and Bengio [6] showed that random search is more efficient than grid search when only a few hyperparameters matter. Bayesian optimization [7] builds a surrogate model of the objective to choose promising configurations. Probst et al. [5] studied Random Forest tuning specifically and found that the number of features tried at each split, the sample fraction and the minimum node size are the most influential hyperparameters.

Population-based metaheuristics such as Genetic Algorithms [10], Differential Evolution [13] and PSO [1] need no gradient, tolerate noisy objectives and handle mixed integer and

continuous search spaces.

0.2.4 Particle Swarm Optimization in Applications

Since its introduction by Kennedy and Eberhart [1], PSO has been widely used for continuous optimization and for tuning machine learning models. Shi and Eberhart [2] added the inertia weight, which controls the balance between exploration and exploitation. Clerc and Kennedy [3] analysed the stability and convergence of the swarm and proposed the constriction factor.

0.2.5 Research Gap

While Random Forests and PSO are both well established, limited published work combines:

- PSO tuning of Random Forests for next-day AQI forecasting in Indian cities
- Official CPCB bulletin AQI combined with reanalysis weather over a long, recent period (2015 to 2026)
- Strictly chronological, leakage-free evaluation with an equal-budget random-search control
- Statistical testing of whether optimization gains are significant

This project addresses these gaps for Delhi.

0.3 Theoretical Background

0.3.1 Time Series Forecasting

A time series is a sequence of observations $\{x_1, x_2, \ldots, x_t\}$ indexed by time. One-step-ahead forecasting predicts the next value from information available up to time t:

$$\hat{y}_{t+1} = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-w+1}; \theta) \tag{1}$$

where w is the look-back window, $\mathbf{x}$ contains AQI history and weather, and θ represents model parameters.

The simplest forecast is *persistence*, which assumes tomorrow equals today:

$$\hat{y}_{t+1} = y_t \tag{2}$$

Any useful model must beat this baseline.

0.3.2 National Air Quality Index

The CPCB AQI [17] converts each pollutant concentration C_p into a sub-index by linear interpolation between breakpoints:

$$I_p = \frac{I_{hi} - I_{lo}}{BP_{hi} - BP_{lo}} (C_p - BP_{lo}) + I_{lo} \tag{3}$$

where $BP_{lo} \leq C_p \leq BP_{hi}$ are the concentration breakpoints and I_{lo}, I_{hi} the corresponding index values. The overall AQI is the largest sub-index:

$$AQI = \max_p I_p \tag{4}$$

Table 1: CPCB AQI categories

Category	**AQI range**	**Health impact**
Good	0–50	Minimal impact
Satisfactory	51–100	Minor breathing discomfort to sensitive people
Moderate	101–200	Discomfort to people with lung or heart disease
Poor	201–300	Breathing discomfort to most people on prolonged exposure
Very Poor	301–400	Respiratory illness on prolonged exposure
Severe	401–500	Affects healthy people; serious impact on the ill

0.3.3 Decision Trees

A regression tree recursively splits the feature space. At each node it chooses the feature j and threshold s that minimize the squared error of the two child regions:

$$\min_{j,s} \Big[\sum_{x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2 \Big] \tag{5}$$

A deep tree has low bias but high variance: small changes in data can change its structure.

0.3.4 Random Forest

A Random Forest [4] averages B trees, each trained on a bootstrap sample of the data, with a random subset of features considered at each split:

$$\hat{f}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}) \tag{6}$$

For trees with variance σ^2 and pairwise correlation ρ, the variance of the average is:

$$\mathrm{Var}\left(\hat{f}_{RF}\right) = \rho\, \sigma^2 + \frac{1 - \rho}{B}\, \sigma^2 \tag{7}$$

More trees reduce the second term, and feature and sample sub-sampling reduce ρ. The hyperparameters therefore control the bias–variance trade-off, which is what PSO tunes.

0.3.5 Particle Swarm Optimization Fundamentals

PSO [1] maintains a swarm of particles. Each particle i has a position $\mathbf{x}_i$ (a candidate solution), a velocity $\mathbf{v}_i$, a personal best position $\mathbf{p}_i$ and access to the swarm's global best $\mathbf{g}$.

Algorithm 1: Basic Particle Swarm Optimization

Initialize positions $\mathbf{x}_i$ and velocities $\mathbf{v}_i$ randomly
Evaluate fitness; set $\mathbf{p}_i \leftarrow \mathbf{x}_i$ and $\mathbf{g} \leftarrow \arg\min f(\mathbf{p}_i)$
for iteration $k = 1$ to K **do**
  for each particle i **do**
    Update velocity $\mathbf{v}_i$
    Update position $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$
    Evaluate $f(\mathbf{x}_i)$; update $\mathbf{p}_i$ if improved
  end for
  Update $\mathbf{g}$ to the best personal best
end for
return g

Representation: Each particle encodes one complete set of hyperparameters as a point in a continuous space.

Fitness Function: The quality of a particle is the validation error of the model it encodes. Lower is better.

Velocity Update: Combines momentum, attraction to the personal best and attraction to the global best:

$$\mathbf{v}_i^{k+1} = w\,\mathbf{v}_i^k + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^k) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^k) \tag{8}$$

Position Update:

$$\mathbf{x}_i^{k+1} = \mathbf{x}_i^k + \mathbf{v}_i^{k+1} \tag{9}$$

where w is the inertia weight, c_1, c_2 are the cognitive and social coefficients, $\mathbf{r}_1, \mathbf{r}_2 \sim U(0,1)^d$, and $\odot$ is element-wise multiplication.

0.3.6 Evaluation Metrics

We use multiple metrics for a comprehensive evaluation.

Root Mean Square Error:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{10}$$

Mean Absolute Error:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{11}$$

R-squared:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{12}$$

Mean Absolute Percentage Error:

$$MAPE = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{13}$$

Category metrics: Both the observed and predicted AQI are mapped to the six CPCB categories, and accuracy, macro precision, macro recall and macro F1 are computed:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \tag{14}$$

Alert AUC: The area under the ROC curve for detecting days with AQI above 300 (*Very Poor* or worse), using the predicted AQI as the score.

0.4 Nature-Inspired Algorithms: Detailed Study

0.4.1 Overview of Nature-Inspired Optimization

Nature-inspired algorithms mimic biological, physical or social phenomena to solve hard optimization problems. These metaheuristics offer several advantages over gradient-based methods:

- No requirement for gradient information
- Effective for non-convex, multi-modal objectives
- Robust to noise and discontinuities in the objective
- Natural parallel evaluation of many candidate solutions
- Ability to escape poor local optima

No single optimizer is best for every problem [14], so the choice of algorithm should match the structure of the problem.

0.4.2 Particle Swarm Optimization: In-Depth Analysis

Biological Inspiration

PSO was inspired by the flocking of birds and the schooling of fish [1]. Key concepts:

- **Individual memory:** Each bird remembers the best location it has found (personal

best)

- **Social learning:** Birds share information, so the flock knows the best location found so far (global best)
- **Momentum:** Birds do not change direction instantly; they keep part of their previous velocity
- **Emergence:** No leader is needed; good solutions emerge from simple local rules

Particle Encoding Scheme

Each particle is a 38-dimensional vector: 6 coordinates for Random Forest hyperparameters and 32 feature switches, one per input feature. Every coordinate lives in $[0, 1]$. The hyperparameter coordinates are decoded to the real range, with integer parameters rounded:

$$\theta_j = L_j + u_j\,(U_j - L_j), \qquad u_j \in [0, 1] \tag{15}$$

Encoding in the unit hypercube means that a tree count (hundreds) and a feature fraction (0.1 to 1) move at comparable speeds.

The feature switches use a threshold transfer function, which turns the continuous position into a binary include/exclude decision:

$$s_f = \begin{cases} 1 & \text{if } u_f \geq 0.5 \\ 0 & \text{otherwise} \end{cases}, \qquad f = 1, \ldots, 32 \tag{16}$$

If fewer than 3 switches are on, the 3 features with the highest u_f are kept, so a particle never describes an empty model. Because the velocity is clamped at 0.2, a switch can only flip after the particle has moved consistently in one direction, which keeps the feature subset stable from one iteration to the next.

Table 2: Particle structure for Random Forest optimization

Dimension	**Hyperparameter**	**Range**	**Type**
1	n_estimators (number of trees)	[50, 300]	Integer
2	max_depth (maximum tree depth)	[3, 30]	Integer
3	min_samples_split	[2, 20]	Integer
4	min_samples_leaf	[1, 15]	Integer
5	max_features (fraction per split)	[0.1, 1.0]	Continuous
6	max_samples (bootstrap fraction)	[0.4, 1.0]	Continuous
7–38	Feature switches (one per feature)	{0, 1}	Binary (threshold)

Inertia Weight

Linearly decreasing inertia: Used in our implementation [2]

$$w_k = w_{start} - (w_{start} - w_{end})\,\frac{k}{K-1} \tag{17}$$

- Large w early (0.9): particles keep momentum and explore widely
- Small w late (0.4): particles slow down and refine around the best region

Alternative Methods:

- Constant inertia: a single fixed w
- Constriction factor [3]: velocity scaled by χ to guarantee convergence
- Adaptive inertia: w adjusted from swarm diversity or success rate

Velocity Clamping

Without limits, velocities can grow until particles fly out of the search space. Each component is clamped:

$$v_{ij} \leftarrow \min\big(\max(v_{ij}, -v_{max}),\, v_{max}\big), \qquad v_{max} = 0.2 \tag{18}$$

Boundary Handling

Reflection: A particle that crosses a boundary is reflected back inside and its velocity component is reversed:

$$x_{ij} \leftarrow \begin{cases} -x_{ij} & \text{if } x_{ij} < 0 \\ 2 - x_{ij} & \text{if } x_{ij} > 1 \end{cases}, \qquad v_{ij} \leftarrow -v_{ij} \tag{19}$$

Simple clipping would pile particles up on the walls and reduce diversity, so reflection is used.

Convergence Analysis

PSO convergence depends on:

- Swarm size: larger swarms explore more but cost more evaluations
- Inertia weight: high inertia explores, low inertia exploits
- Acceleration coefficients: c_1 favours individual search, c_2 favours collective search

Clerc and Kennedy [3] showed that particle trajectories are stable when

$$0 < c_1 + c_2 < 4\,(1 + w), \qquad |w| < 1 \tag{20}$$

Our settings ($w \leq 0.9$, $c_1 + c_2 = 3$) satisfy this condition.

0.4.3 Genetic Algorithm

GA [10] evolves a population of chromosomes using selection, crossover and mutation inspired by natural evolution. Gaussian mutation, for example:

$$gene' = gene + \mathcal{N}(0, \sigma^2) \tag{21}$$

Advantages: Flexible encoding, strong global search

Disadvantages: Many operators and parameters to choose, slower convergence on continuous problems

0.4.4 Ant Colony Optimization

ACO [11] models the foraging of ants using pheromone trails. The probability of moving from node i to j is:

$$p_{ij} = \frac{[\tau_{ij}]^{\alpha}[\eta_{ij}]^{\beta}}{\sum_{k \in allowed}[\tau_{ik}]^{\alpha}[\eta_{ik}]^{\beta}} \tag{22}$$

Applications: Routing, scheduling, feature selection; best suited to discrete, path-like problems

0.4.5 Simulated Annealing

SA [12] mimics metallurgical annealing. A worse solution is accepted with probability:

$$P(accept) = e^{-\Delta E/T} \tag{23}$$

where T decreases over time.

Advantages: Simple, escapes local minima

Disadvantages: Single-solution search, slow convergence, sensitive to the cooling schedule

0.4.6 Differential Evolution

DE [13] creates mutants from vector differences:

$$\mathbf{v}_i = \mathbf{x}_{r1} + F\left(\mathbf{x}_{r2} - \mathbf{x}_{r3}\right) \tag{24}$$

Advantages: Very effective for continuous optimization

Disadvantages: Needs a reasonably large population

0.4.7 Why PSO for Random Forest Optimization?

We chose PSO over the alternatives because:

1. **Expensive fitness:** Each evaluation trains five forests. PSO shares information between particles, so each new evaluation is guided by all earlier ones.
2. **Continuous encoding:** The unit-hypercube encoding suits PSO's velocity model and handles both integer and continuous hyperparameters.
3. **Few control parameters:** Only w, c_1, c_2, swarm size and iterations, compared with GA's selection, crossover and mutation choices.
4. **No gradient:** The validation error is noisy and stepped because of integer rounding, which gradient methods cannot handle.
5. **Parallel evaluation:** All particles in an iteration can be evaluated independently.

0.4.8 Hybrid Approaches

Recent research explores hybrid algorithms:

- GA-PSO: genetic operators applied to the swarm to maintain diversity
- PSO with local search: gradient-free local refinement around the global best
- Surrogate-assisted PSO: a cheap model of the fitness function reduces the number of expensive evaluations

Future work may explore these approaches.

0.5 Methodology and Implementation

0.5.1 System Architecture

Our system comprises five major components:

1. Data Collection Module: CPCB AQI bulletins and ERA5 weather
2. Preprocessing and Feature Engineering Pipeline
3. Random Forest Regression Model
4. Particle Swarm Optimization Framework (hyperparameters and feature selection)
5. Evaluation and Visualization Suite

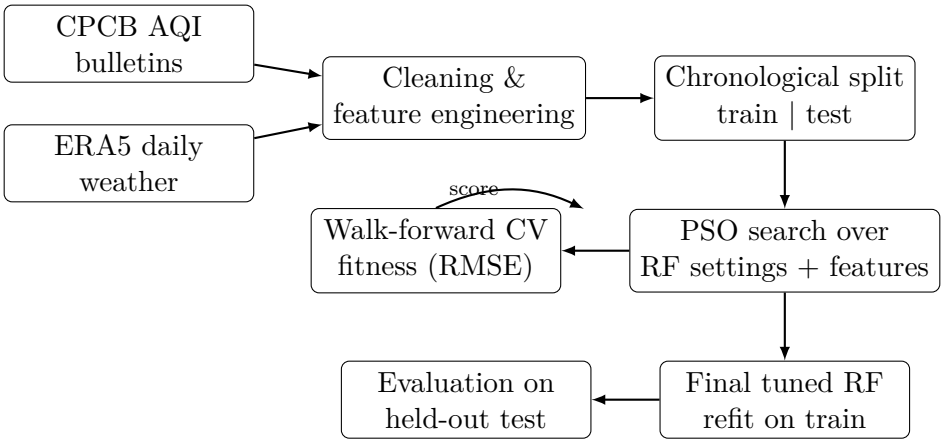


Figure 1: System architecture of the PSO-optimized Random Forest pipeline

0.5.2 Data Collection

City Selection

Delhi was selected because:

- It is the most polluted major city in the CPCB network, with strong seasonal swings
- Its daily AQI record is long and nearly complete (26 missing days in 11.5 years)
- Forecasts there have the largest public-health value

Data Sources and Period

Table 3: Data sources

Source	**Variables**	**Coverage**
CPCB daily AQI bulletins [17, 19]	City AQI, category, prominent pollutant	1 May 2015 – 25 Oct 2026; 4,170 reported days
ERA5 reanalysis via Open-Meteo [18, 20]	Temperature, rain, humidity, wind, pressure, radiation, dew point	1 Apr 2015 – 25 Oct 2026; complete

The bulletin AQI is the official city value published by CPCB each day, averaged over Delhi's monitoring stations. The weather is taken at central Delhi (28.61 N, 77.21 E).

This period captures several distinct regimes:

- High-pollution years (2015–2019)
- The COVID-19 lockdown and recovery (2020–2021)
- Recent years under the National Clean Air Programme (2022–2026)

Data Validation

The bulletin AQI was cross-checked against the independent Kaggle "Air Quality Data in India (2015–2020)" dataset over their 1,856 common days. The two series have a correlation of 0.927,

which confirms that the bulletin values are consistent.

0.5.3 Data Preprocessing

Missing Value Treatment

- AQI: 26 of 4,196 calendar days (0.6%) have no bulletin
- Weather: no missing values
- Input features: forward fill, which uses only the last value known at prediction time
- Target: never imputed; the 26 days with no observed AQI are removed as targets

Linear interpolation was avoided on purpose: it would use the next observation and leak future information into the features.

Normalization

No scaling is applied to the Random Forest, because tree splits do not depend on feature scale. The linear and kernel baselines use standard scaling fitted on the training period only:

$$z = \frac{x - \mu_{train}}{\sigma_{train}} \tag{25}$$

0.5.4 Feature Engineering

All features for day t use only information available at the end of day t. A total of 32 features are created.

AQI History Features:

- Current AQI and lags: $AQI_t, AQI_{t-1}, AQI_{t-2}, AQI_{t-3}, AQI_{t-7}$
- Rolling means over 3, 7, 14 and 30 days
- 7-day rolling standard deviation
- Day-on-day change: $\Delta AQI_t = AQI_t - AQI_{t-1}$

$$\overline{AQI}_n(t) = \frac{1}{n}\sum_{i=0}^{n-1} AQI_{t-i} \tag{26}$$

Weather Features:

- Temperature (maximum, minimum, mean), dew point, relative humidity
- Rainfall, and 3-day rainfall total
- Wind speed (mean, maximum) and dominant wind direction
- Surface pressure and shortwave radiation

- Day-on-day change in minimum temperature, pressure, dew point and wind speed

Wind direction is circular, so it is encoded as sine and cosine:

$$(\sin\phi,\ \cos\phi), \qquad \phi = \text{direction in radians} \tag{27}$$

Calendar Features: The day of year and day of week of the *forecast* day are encoded cyclically:

$$doy_{sin} = \sin\left(\frac{2\pi\, d}{365.25}\right), \qquad doy_{cos} = \cos\left(\frac{2\pi\, d}{365.25}\right) \tag{28}$$

This makes 31 December and 1 January neighbours.

0.5.5 Random Forest Model Architecture

Model Design

- Input: 32 features for day t
- Ensemble: B regression trees (CART), each on a bootstrap sample
- Split criterion: squared error
- Output: the average of all tree predictions, the forecast AQI for day $t + 1$

Why Random Forest

- Captures non-linear effects and interactions between weather and pollution
- Robust to correlated inputs such as lags and rolling means
- Needs no feature scaling
- Works well with about 3,000 training samples, where deep networks overfit easily
- Provides feature importances for interpretation

0.5.6 PSO Implementation

Particle Representation

Each particle is a 38-dimensional vector in the unit hypercube:

$$\mathbf{x} = [u_{trees}, u_{depth}, u_{split}, u_{leaf}, u_{features}, u_{samples}, u_{f_1}, \ldots, u_{f_{32}}] \in [0,1]^{38} \tag{29}$$

Implemented as a NumPy array. The hyperparameter part is decoded by:

```
def decode(position, space=SEARCH_SPACE):
    params = {}
    for u, p in zip(position, space):
```

```
        value = p.low + float(u) * (p.high - p.low)
        params[p.name] = int(round(value)) if p.integer \
                         else round(value, 3)
    return params
```

The feature switches are decoded by:

```
def decode_solution(position, feature_names):
    params = decode(position[:6])
    switches = position[6:]
    keep = switches >= 0.5            # threshold transfer
    if keep.sum() < 3:                # never an empty model
        keep = np.zeros(len(switches), bool)
        keep[np.argsort(-switches)[:3]] = True
    return params, [f for f, k in zip(feature_names, keep) if k]
```

Swarm Initialization

A swarm of 12 particles is created with:

- Positions drawn uniformly from $[0, 1]^{38}$, so each feature starts switched on with probability 0.5
- Velocities drawn uniformly from $[-v_{max}, v_{max}]^{38}$
- A seeded random number generator, so every run is reproducible

Fitness Evaluation

The fitness of a particle is the mean RMSE over 5 expanding-window folds of the training period, using only the features the particle switches on:

$$F(\mathbf{x}) = \frac{1}{5}\sum_{k=1}^{5} RMSE\big(RF_{\theta(\mathbf{x})} \text{ trained on folds } 1..k,\ \text{validated on fold } k+1\big) \tag{30}$$

The model is always trained on earlier days and validated on later days. Identical decoded solutions (the same hyperparameters and the same feature subset) are cached, so the same configuration is never trained twice.

Velocity and Position Update

```
def update_velocity(self, v, x, pbest, gbest, w):
    r1 = self.rng.random(x.shape)
    r2 = self.rng.random(x.shape)
    cognitive = self.cfg.c1 * r1 * (pbest - x)
    social = self.cfg.c2 * r2 * (gbest - x)
```

```
    return np.clip(w * v + cognitive + social,
                   -self.cfg.v_max, self.cfg.v_max)

def update_position(x, v):
    x_new = x + v
    for _ in range(2):          # reflect off the walls
        below, above = x_new < 0.0, x_new > 1.0
        x_new = np.where(below, -x_new, x_new)
        x_new = np.where(above, 2.0 - x_new, x_new)
        v = np.where(below | above, -v, v)
    return np.clip(x_new, 0.0, 1.0), v
```

Best Updates

After each evaluation:

$$\mathbf{p}_i \leftarrow \mathbf{x}_i \;\text{ if } F(\mathbf{x}_i) < F(\mathbf{p}_i), \qquad \mathbf{g} \leftarrow \arg\min_i F(\mathbf{p}_i) \tag{31}$$

Termination Criteria

The algorithm stops after 25 iterations, which gives $12 \times (25 + 1) = 312$ fitness evaluations per run.

0.5.7 Training Procedure

Data Splitting

- Training: 3,115 days (30 May 2015 – 31 December 2023), used for model fitting and all PSO tuning
- Testing: 1,026 days (1 January 2024 – 24 October 2026), used only for final evaluation

Temporal ordering is preserved, with no shuffling. The first 29 days are dropped because the 30-day rolling mean is incomplete.

Optimization Process

For each of 5 independent PSO runs (seeds 0 to 4):

Initialize the swarm
for each iteration **do**
    Decode every particle into RF hyperparameters
    Train RF on each walk-forward fold and record mean validation RMSE
    Update personal bests, global best, velocities and positions
end for
Record the best configuration, its feature subset and its validation RMSE

The run with the lowest validation RMSE is selected; this rule was fixed before any test result was seen. Its hyperparameters and feature subset are refit on the full training period and evaluated once on the test period.

0.5.8 Implementation Details

Technology Stack

- Python: 3.11
- scikit-learn: 1.9 (Random Forest, metrics, baselines)
- NumPy 2.4 and pandas 3.1: data handling and the PSO implementation
- SciPy 1.17: statistical tests
- Matplotlib 3.11: visualization
- joblib: parallel execution of independent runs
- pytest: automated tests

Computational Resources

- CPU: Apple M5 Pro, 15 cores
- RAM: 24 GB
- GPU: not required

Reproducibility

Ensured through:

- Fixed random seeds for PSO, random search and every forest
- Configuration in a single file (`aqipso/config.py`)
- Version-pinned requirements
- 16 automated tests, including checks that no feature uses future data and that the feature switches decode correctly

0.6 Experimental Setup and Test Functions

0.6.1 Experimental Configuration

PSO Parameters

Table 4: PSO configuration parameters

Parameter	**Value**
Swarm size	12 particles
Number of iterations	25
Fitness evaluations per run	312
Inertia weight w	0.9 decreasing linearly to 0.4
Cognitive coefficient c_1	1.5
Social coefficient c_2	1.5
Particle dimensions	38 (6 hyperparameters + 32 feature switches)
Feature switch threshold	0.5 (at least 3 features kept)
Velocity limit v_{max}	0.2 (per dimension)
Boundary handling	Reflection
Independent runs	5 (seeds 0–4)

Random Forest Evaluation Parameters

Table 5: Model training and evaluation configuration

Parameter	**Value**
Split criterion	Squared error
Fitness function	Mean RMSE over 5 walk-forward folds
Training period	30 May 2015 – 31 Dec 2023 (3,115 days)
Test period	1 Jan 2024 – 24 Oct 2026 (1,026 days)
Forest random seed	42 (search); 100–109 (robustness check)

Hyperparameter Search Space

The search space is shown in Table 2 (Section 0.4.2). It covers forests from 50 to 300 trees, depths from 3 to 30, and feature and sample fractions from very small to complete.

0.6.2 Baseline Models

To validate the effectiveness of PSO optimization, we compare against:

Baseline 1: Persistence

Tomorrow's AQI is predicted to equal today's ($\hat{y}_{t+1} = y_t$). This is the minimum any useful model must beat.

Baseline 2: Default Random Forest

A Random Forest with scikit-learn's default hyperparameters:

- Trees: 100

- Maximum depth: unlimited
- Minimum samples to split: 2; minimum samples per leaf: 1
- Features per split: all (1.0); bootstrap sample: all days

Baseline 3: Random Search

Random selection as a control for PSO:

- 312 random points in the same 38-dimensional space, the same budget as PSO
- Same decoding, same feature switches and same fitness function
- 5 independent runs; best configuration selected

Ablation: Hyperparameter-Only Search

To measure what feature selection adds, PSO and random search were also run over the 6 hyperparameters alone, with all 32 features always used (5 runs each, same budget). The best hyperparameter-only PSO model is also evaluated on the test period.

Baseline 4: Traditional Models

Untuned models trained on the same 32 features:

- Ridge regression ($\alpha = 1$, standardized inputs)
- Support Vector Regression (RBF kernel, $C = 100$, $\epsilon = 5$)
- Gradient Boosting Regressor (default settings)

0.6.3 Test Functions and Benchmarks

Optimization Test Functions

To validate the PSO implementation, it was first tested on standard benchmarks in 6 dimensions (30 particles, 200 iterations, 5 seeds).

Sphere Function:

$$f(\mathbf{x}) = \sum_{i=1}^{n} x_i^2, \qquad x_i \in [-100, 100] \tag{32}$$

Global minimum: $f(0, \ldots, 0) = 0$. Unimodal; tests convergence speed.

Rastrigin Function:

$$f(\mathbf{x}) = 10n + \sum_{i=1}^{n} \left[x_i^2 - 10\cos(2\pi x_i) \right], \qquad x_i \in [-5.12, 5.12] \tag{33}$$

Highly multi-modal; tests escape from local minima.

Rosenbrock Function:

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right], \qquad x_i \in [-5, 5] \tag{34}$$

Tests convergence along a narrow curved valley.

Table 6: PSO results on benchmark functions (6 dimensions, 5 runs; global minimum 0)

Function	Mean best f	Std Dev	Best run
Sphere	1.9×10^{-17}	3.2×10^{-17}	1.0×10^{-18}
Rastrigin	2.99	1.41	1.99
Rosenbrock	1.10	0.19	0.84

PSO solves the Sphere function to machine precision, which confirms that the operators are implemented correctly. On Rastrigin it reaches low values but stops in a local minimum near $f \approx 2$, with two coordinates trapped in the neighbouring wells at ± 1. On Rosenbrock it reaches the valley quickly but advances slowly along it. These are the expected strengths and weaknesses of global-best PSO.

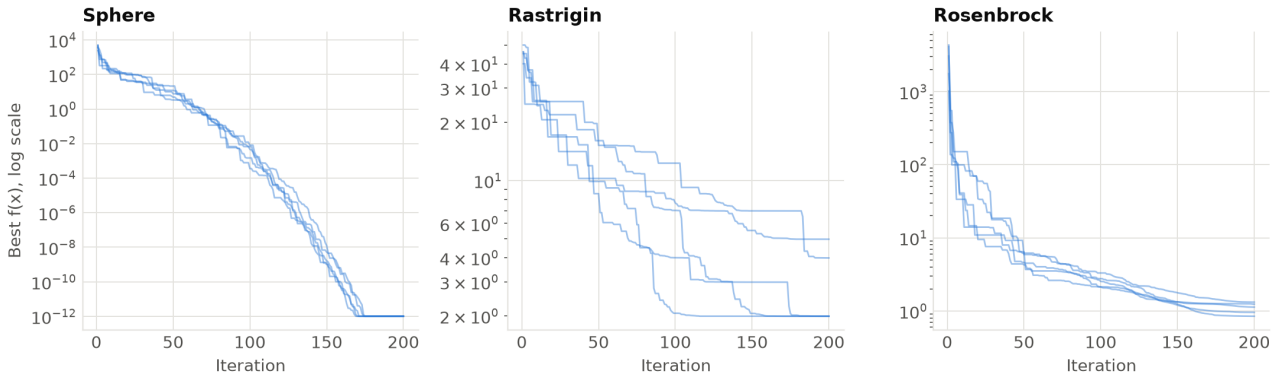


Figure 2: PSO convergence on the three benchmark functions (5 runs each, log scale)

0.6.4 Cross-Validation Strategy

Time Series Cross-Validation

For PSO fitness evaluation, walk-forward validation with an expanding window is used:

- 5 temporal folds over the training period
- Each fold trains on all earlier days and validates on the next block of 519 days
- The first fold trains on 520 days; the last trains on 2,596 days
- Fitness = average RMSE across the 5 folds

Shuffled k-fold cross-validation was not used, because it would let the model train on days after the ones it is validated on.

Hold-Out Test

The test period (2024 to 2026) is never used during the search. It is used once, to evaluate the final selected model.

0.6.5 Sensitivity Analysis

PSO Parameter Sensitivity

The effect of PSO's own parameters was tested on the 6-dimensional Rastrigin function, with 100 iterations and 10 seeds per variant:

Table 7: PSO parameter sensitivity on Rastrigin (lower is better)

Variant	**Mean best**	**Std Dev**	**Best run**
Base: 20 particles, w 0.9→0.4, $c_1 = c_2 = 1.5$	6.63	2.61	2.99
10 particles	5.40	2.78	1.04
40 particles	3.59	2.11	1.00
Constant inertia $w = 0.7$	3.73	1.84	1.99
Constant inertia $w = 0.4$	5.27	2.57	1.00
$c_1 = 2.0$, $c_2 = 1.0$ (more cognitive)	4.77	3.42	1.00
$c_1 = 1.0$, $c_2 = 2.0$ (more social)	3.76	2.39	1.03

Results show high run-to-run variance on this multi-modal function: standard deviations are 2 to 3 points, so differences of 1 to 2 points between variants are within noise. A larger swarm (40 particles) gave the lowest mean, as expected from its broader exploration. With only 100 iterations, the decreasing-inertia schedule spends much of the run exploring and did not beat constant $w = 0.7$; with the 25-iteration budget of the main experiment, the choice of schedule matters less than the swarm's ability to locate the right region.

0.6.6 Statistical Significance Testing

Block Bootstrap

Daily forecast errors are autocorrelated, so ordinary resampling would overstate confidence. A moving-block bootstrap [15] resamples whole weeks (block length 7, 5,000 resamples) and computes the RMSE gain:

$$\Delta = RMSE_{baseline} - RMSE_{tuned} \tag{35}$$

The 2.5th and 97.5th percentiles of Δ give a 95% confidence interval.

Paired t-test

Compares the daily squared errors of the default and tuned models:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \tag{36}$$

where $\bar{d}$ is the mean difference and s_d the standard deviation of differences.

Wilcoxon Signed-Rank Test

A non-parametric alternative for non-normal differences:

$$W = \sum_{i=1}^{n} \text{sgn}(x_{2,i} - x_{1,i}) \cdot R_i \tag{37}$$

0.6.7 Computational Efficiency Metrics

Training Time Analysis

Measured for each approach:

- Wall-clock time per search run
- Number of fitness evaluations, and how many were unique after caching
- Iteration at which the global best stopped improving

The 20 search runs (PSO and random search, each with and without feature selection, 5 seeds each) were executed in parallel on 15 CPU cores.

0.7 Results and Discussion

0.7.1 PSO Convergence Analysis

Fitness Evolution

Figure 3 shows the global-best validation RMSE across iterations for all five PSO runs of the joint (hyperparameter and feature) search:

- Iteration 1: mean global best = 41.22 (already below the default forest's 41.52)
- Iteration 10: mean global best = 40.20
- Iteration 25: mean global best = 39.94; best run = 39.73

The joint search keeps improving for longer than a hyperparameter-only search, because there are many more feature subsets to explore. The global best was still improving at iterations 22 to 25 in every run, so a larger budget would probably lower the validation error further.

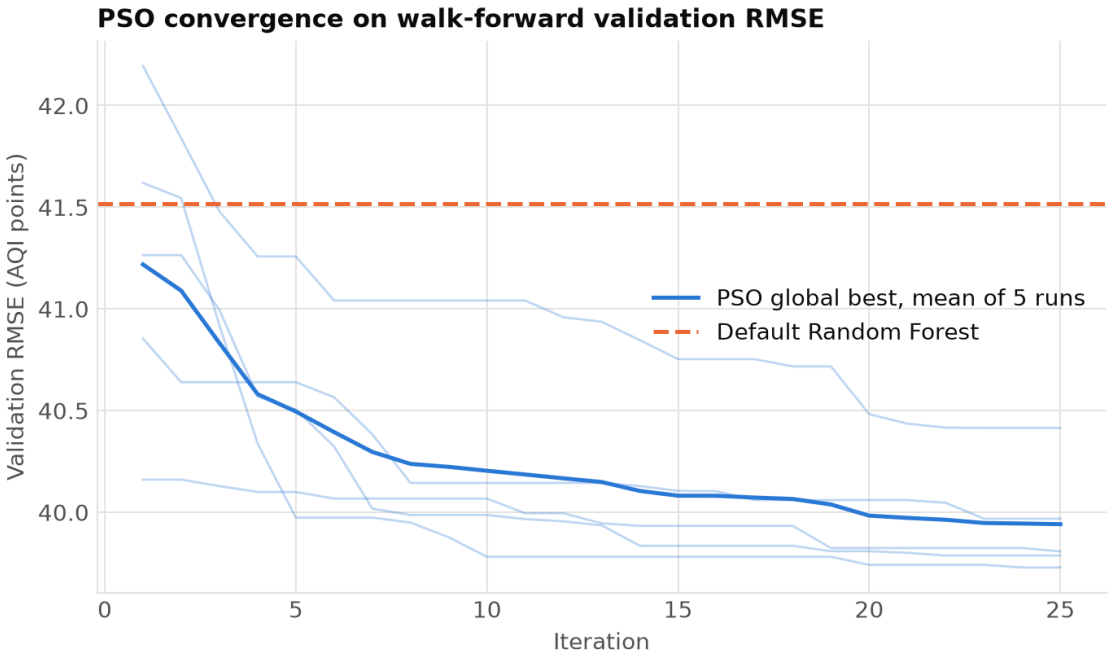


Figure 3: PSO convergence on walk-forward validation RMSE (5 runs)

Swarm Diversity

Diversity is measured as the mean distance of particles from the swarm centre:

$$D_k = \frac{1}{N} \sum_{i=1}^{N} \left\| \mathbf{x}_i^k - \bar{\mathbf{x}}^k \right\| \tag{38}$$

In the selected run, diversity fell from 1.28 to 0.19 while the swarm's mean RMSE fell from 42.97 to 39.85, showing the expected shift from exploration to exploitation as the inertia weight decreased. Diversity is larger than in a 6-dimensional search simply because distances are measured in 38 dimensions.

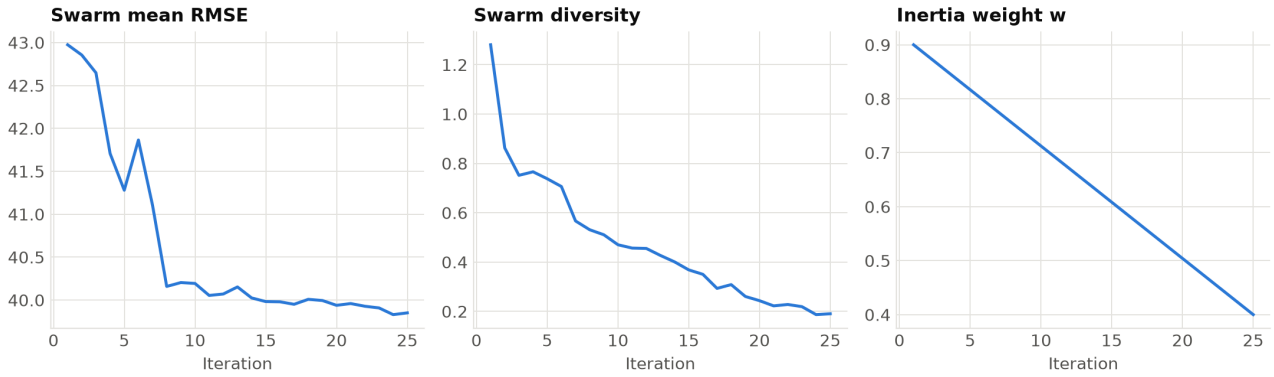


Figure 4: Swarm mean fitness, diversity and inertia weight in the selected run

0.7.2 Optimal Configuration

The run with the lowest validation RMSE (seed 0, RMSE 39.73) was selected.

Table 8: Best configuration found by PSO

Setting	Default	PSO, hyperparameters only	PSO, joint (final)
n_estimators	100	291	224
max_depth	Unlimited	17	18
min_samples_split	2	6	8
min_samples_leaf	1	3	2
max_features	1.0	0.797	0.799
max_samples	1.0	0.655	0.592
Features used	32	32	14
Validation RMSE	41.52	40.66	**39.73**

Key Insights:

- Both searches found very similar hyperparameters: depth near 17, about 80% of features tried per split, and 60 to 65% of days per tree
- Limited depth and leaves of 2 to 3 days regularize each tree; feature and sample subsampling decorrelate the trees
- Feature selection removed 18 of 32 features and lowered validation RMSE by a further 0.93 points

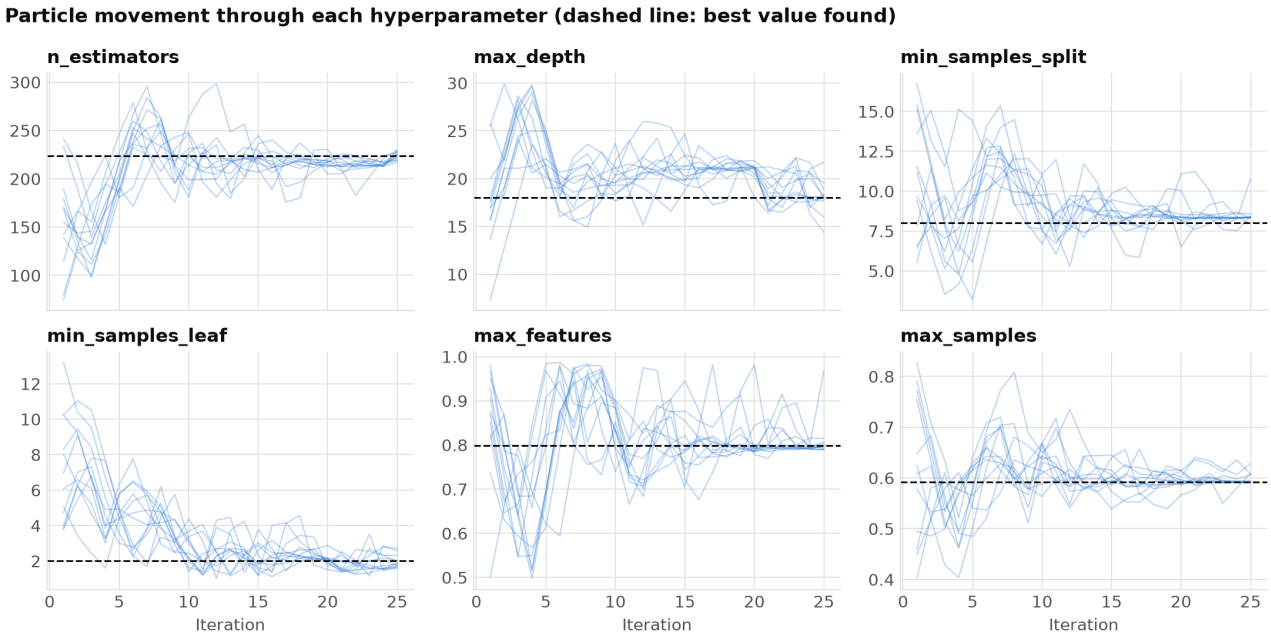


Figure 5: Particle trajectories through each hyperparameter in the selected run

0.7.3 Feature Selection Results

The 14 features kept by the final model are: today's AQI, yesterday's AQI, the 3-day and 30-day mean AQI, the day-on-day AQI change, maximum temperature, rainfall, 3-day rainfall, maximum wind speed, the change in mean wind speed, wind direction (sine and cosine),

shortwave radiation, and one day-of-week term.

Consistent choices across the five runs:

- **Always kept (5 of 5 runs):** today's AQI, 3-day rainfall, shortwave radiation, wind direction (sine), change in mean wind speed, day of week (cosine)
- **Never kept (0 of 5 runs):** relative humidity, surface pressure, dew point, and the AQI two days ago
- The runs kept between 12 and 18 features

The kept variables are those that physically control pollution: wind and its changes disperse it, rain washes it out, and solar radiation drives the daytime mixing layer. Humidity, pressure and dew point are strongly correlated with temperature and season, so they add little once those are present. Recent AQI is represented by today's value and short averages, which makes the separate lags redundant.

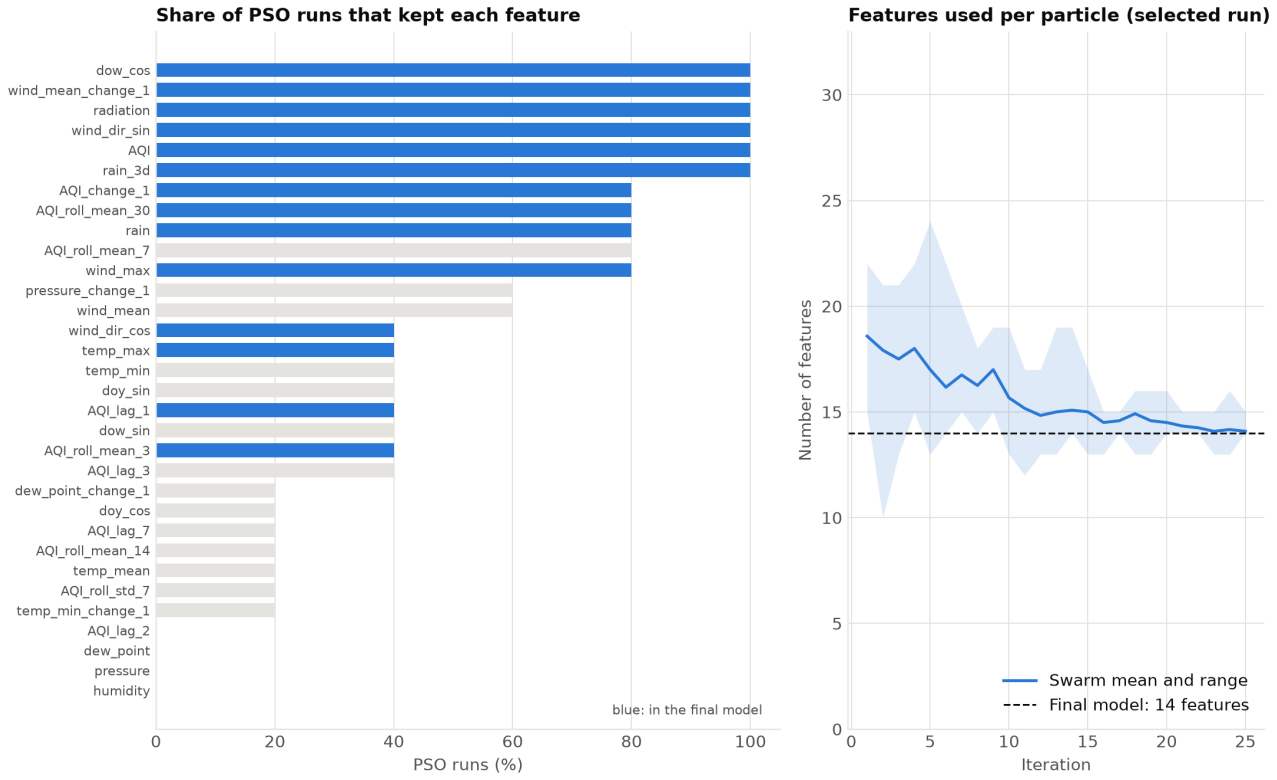


Figure 6: Share of PSO runs that kept each feature (left) and the number of features used by the swarm in the selected run (right)

0.7.4 Prediction Performance

Test Period Results

Table 9: Test performance, 1 Jan 2024 – 24 Oct 2026 (1,026 days)

Metric	Persistence	Default RF	PSO, hyp. only	PSO, joint (final)
RMSE	40.21	33.85	**33.57**	33.77
MAE	28.97	25.38	**25.14**	25.15
Median absolute error	21.00	19.39	18.87	**18.08**
MAPE (%)	17.16	15.89	15.74	**15.65**
R^2	0.832	0.881	**0.883**	0.882
Category accuracy	0.688	**0.719**	0.715	0.714
Category precision (macro)	0.547	0.607	0.611	**0.622**
Category recall (macro)	0.548	**0.555**	0.552	0.552
Category F1 (macro)	0.548	0.568	0.566	**0.569**
AUC, AQI > 300	0.978	0.986	**0.987**	0.986

Analysis:

- The final model beats the default forest on RMSE, MAE, median error, MAPE, R^2, category precision and F1, but by small margins (RMSE 0.25%, median error 6.8%)
- It is slightly worse than the hyperparameter-only PSO model on RMSE (33.77 against 33.57), even though its validation error was much lower
- All forests beat persistence substantially (RMSE 16.0% lower for the final model)
- All forests detect *Very Poor* or worse days very well (AUC 0.986 to 0.987)

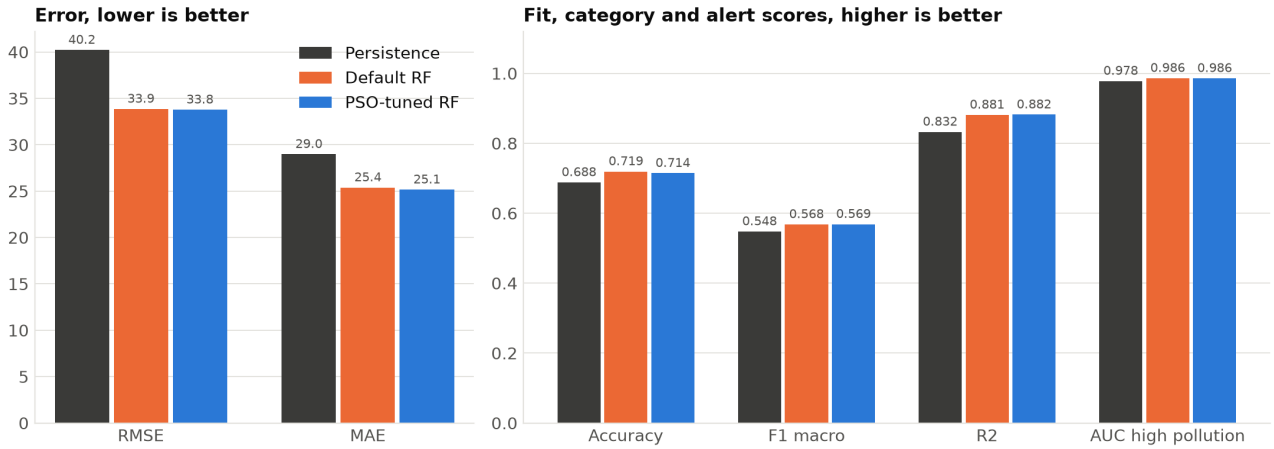


Figure 7: Test metrics of persistence, the default forest and the final PSO model

0.7.5 Temporal Performance Analysis

Performance Across Years and Seasons

Table 10: Test RMSE by period

Period	Days	Mean AQI	Persist.	Default RF	PSO hyp.	PSO joint
2024	365	208.6	41.98	**34.12**	34.20	34.30
2025	365	201.0	38.38	34.23	**33.66**	34.07
2026 (to 24 Oct)	296	166.6	40.17	33.04	**32.67**	32.72
Winter (Oct–Feb)	385	281.6	48.29	40.22	**40.16**	40.22
Rest of year (Mar–Sep)	641	141.0	34.45	29.37	**28.90**	29.21

Errors are largest in the winter smog season, when AQI is highest and most volatile. In every period the three forests are within 0.6 points of each other, while persistence is 4 to 8 points worse.

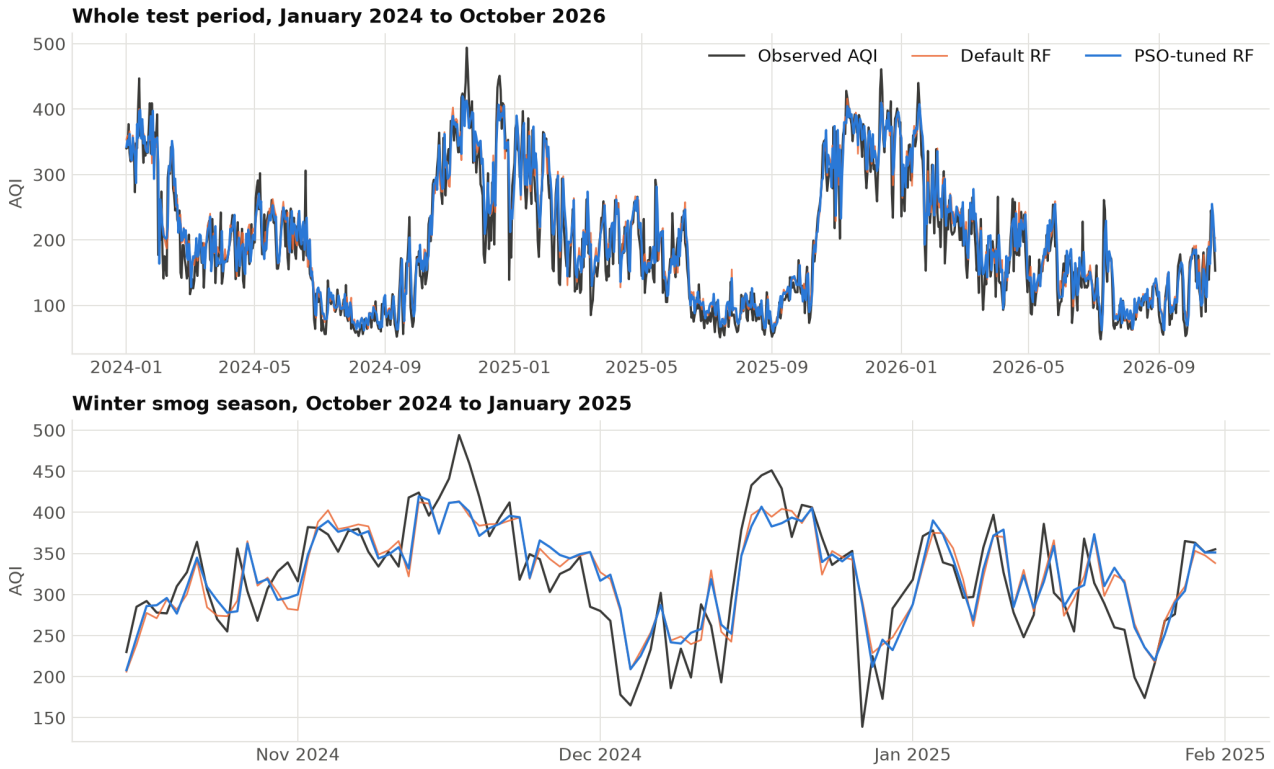


Figure 8: Observed and forecast AQI over the test period, with the 2024–25 winter in detail

0.7.6 Comparison with Baseline Models

Performance Comparison

Table 11: Model comparison on the test period

Model	**RMSE**	**MAE**	R^2
Persistence	40.21	28.97	0.832
Default Random Forest	33.85	25.38	0.881
PSO Random Forest, joint (final)	**33.77**	**25.15**	**0.882**
Gradient Boosting (default)	33.61	25.33	0.883
PSO Random Forest, hyperparameters only	33.57	25.14	0.883
Support Vector Regression	33.46	25.40	0.884
Ridge regression	33.45	25.27	0.884

Key Findings:

- All learned models fall within 0.4 RMSE points of each other and beat persistence by 16 to 17%
- Ridge regression and SVR have the lowest RMSE. With good lag and weather features, next-day AQI is close to a linear problem
- The final PSO model uses less than half the features of every other model at very similar accuracy

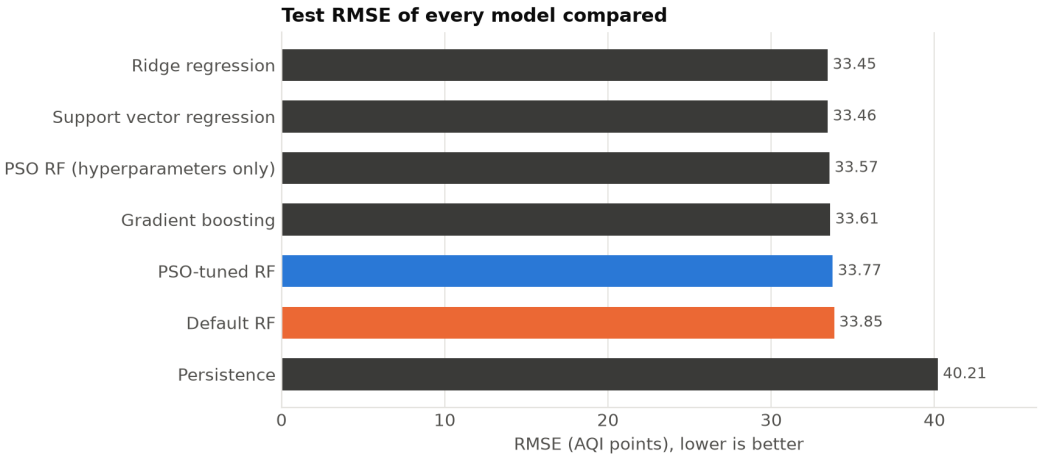


Figure 9: Test RMSE of every model

PSO versus Random Search

Table 12: PSO and random search with the same 312-evaluation budget (RMSE, 5 runs each)

Method	Best val.	Mean val.	Mean test
Default RF	41.52	41.52	33.81
Random search, hyperparameters only	40.75	40.82	33.62
PSO, hyperparameters only	40.66	40.70	33.55
Random search, joint	40.04	40.54	33.96
PSO, joint	**39.73**	**39.94**	**33.47**

Test RMSE is averaged over 10 forest seeds and over the five runs of each method. In both search spaces PSO beat random search on mean validation and mean test error. The advantage is larger in the 38-dimensional joint space (0.60 validation points) than in the 6-dimensional space (0.12), because unguided sampling becomes much less effective as the number of dimensions grows. Four of the five joint PSO runs beat the best joint random-search run. Over its five runs, joint PSO also has the lowest mean test RMSE of any search (33.47, 1.0% below the default forest).

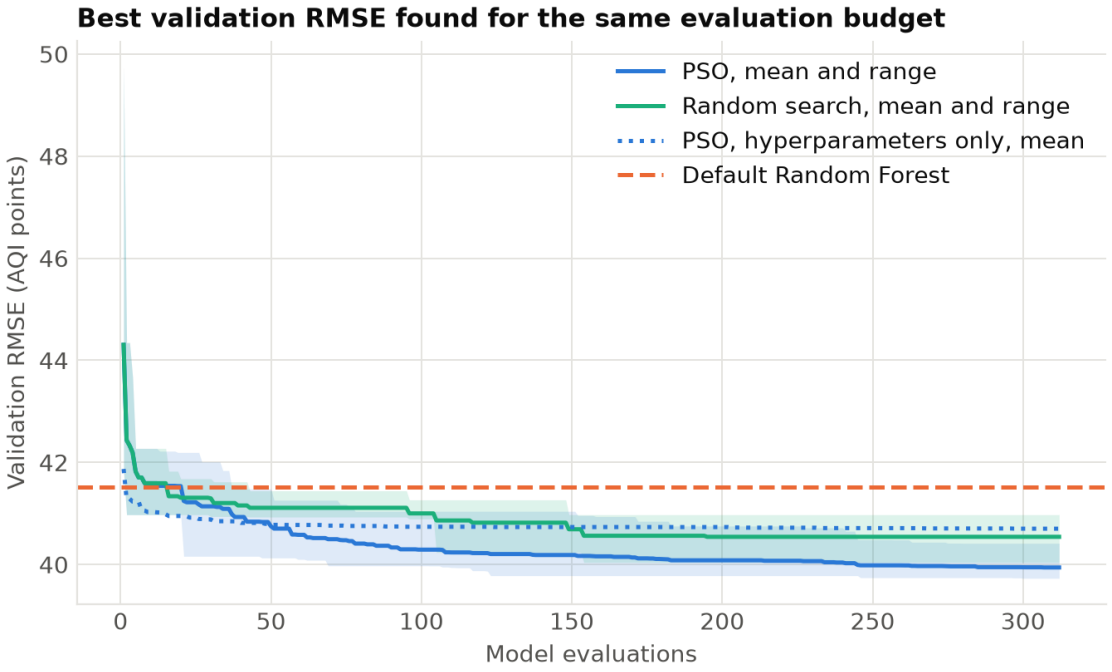


Figure 10: Best validation RMSE found by PSO and random search for the same evaluation budget

Statistical Significance

Comparison of the final PSO model with the default forest on the test period:

- Block-bootstrap RMSE gain: 0.09 AQI points, 95% CI −0.41 to 0.59 (positive in 61.7% of resamples)

- Paired t-test on squared errors: $t = 0.31$, $p = 0.76$
- Wilcoxon signed-rank test: $p = 0.66$
- Final model more accurate on 49.9% of days
- Against persistence: gain 6.44 points, 95% CI 4.85 to 8.06 ($p < 0.001$)
- Against the hyperparameter-only PSO model: -0.20 points, 95% CI -0.67 to 0.26 (t-test $p = 0.40$)

The improvement over persistence is highly significant. The differences between the default forest and the two PSO models are not statistically significant.

0.7.7 Hyperparameter Importance Analysis

Correlation with Performance

Spearman correlation between each setting and validation RMSE, over all 1,560 unique configurations evaluated across the five joint PSO runs (negative means larger values lower the error):

Table 13: Setting–fitness correlation

Setting	**Spearman ρ with RMSE**
min_samples_leaf	0.659
max_depth	−0.346
max_samples	0.344
max_features	−0.336
n_estimators	−0.304
min_samples_split	0.221
Number of features used	−0.138

Leaf size has the strongest effect: large leaves underfit, and the best runs used leaves of 2 to 5 days. More trees, deeper trees and more features per split all helped. The weak negative correlation for the number of features shows that the choice of *which* features matters more than how many.

0.7.8 Feature Importance Results

Table 14: Top 10 most important features of the final model (impurity-based)

Feature	Importance
AQI today (AQI_t)	0.635
3-day mean AQI	0.176
AQI yesterday	0.032
Maximum wind speed	0.029
30-day mean AQI	0.024
Change in mean wind speed	0.019
Shortwave radiation	0.017
Rainfall	0.016
Wind direction (sine)	0.014
3-day rainfall	0.009

Today's AQI dominates, and AQI history accounts for 88% of total importance. Wind, rain and radiation are the most useful weather variables: they control how quickly pollution disperses or is washed out.

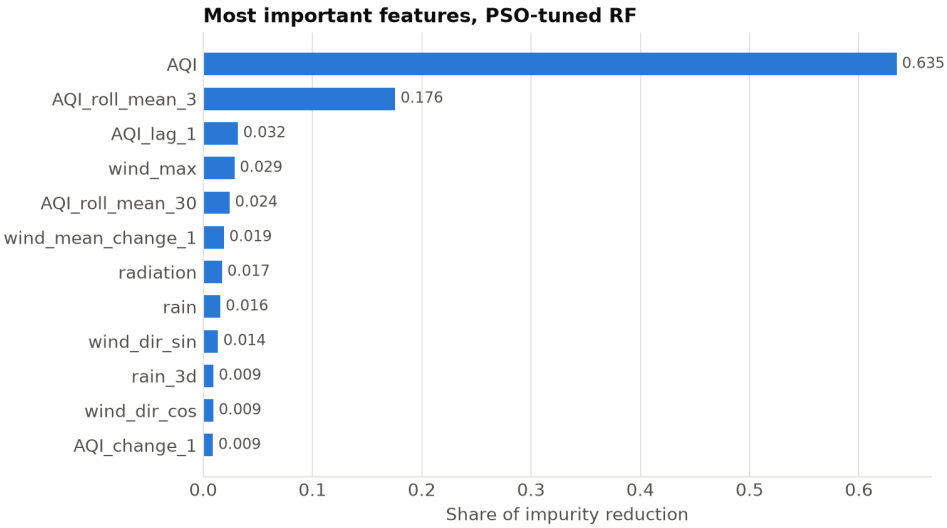


Figure 11: Feature importance of the final PSO model

0.7.9 Error Analysis

Residual Distribution

Residuals (observed minus predicted) of the final model:

- Mean: -7.72 (the model over-predicts on average)
- Std Dev: 32.89
- Skewness: -0.16 (nearly symmetric)

- Kurtosis: 4.67 (heavier tails than a normal distribution)

Shapiro–Wilk test: $p = 1.3 \times 10^{-10}$, so normality is rejected because of the heavy tails.

The negative mean reflects a downward trend: the test period's mean AQI (193.8) is lower than the training period's (216.9), and the model, trained mostly on dirtier years, forecasts slightly high (mean forecast 201.5).

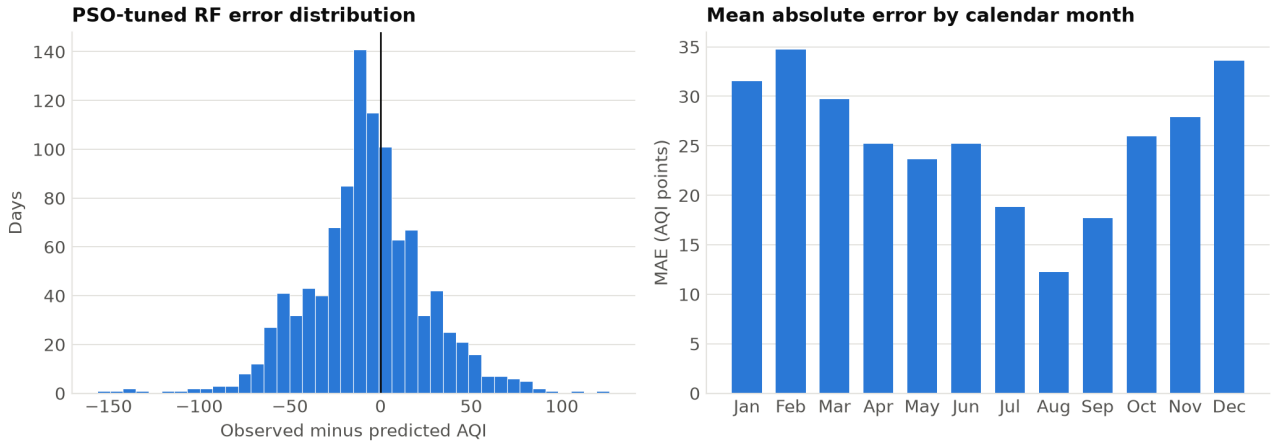


Figure 12: Residual distribution and mean absolute error by month

Lag on Sharp Changes

The forecasts correlate more strongly with the previous day's AQI (0.976) than with the day being forecast (0.942), so sharp changes are followed a day late. On the 28 test days when AQI moved by more than 100 points, MAE was 87.0, against 23.4 on all other days. Such jumps are driven by events the features do not capture, such as sudden stubble-burning plumes, fireworks or a passing western disturbance.

Category Errors

Of the 1,026 test days, 293 were placed in the wrong CPCB category, and 292 of these were only one category away. *Severe* days are rare (27 days) and hard to forecast: 10 were predicted as *Severe*, most of the rest as *Very Poor*.

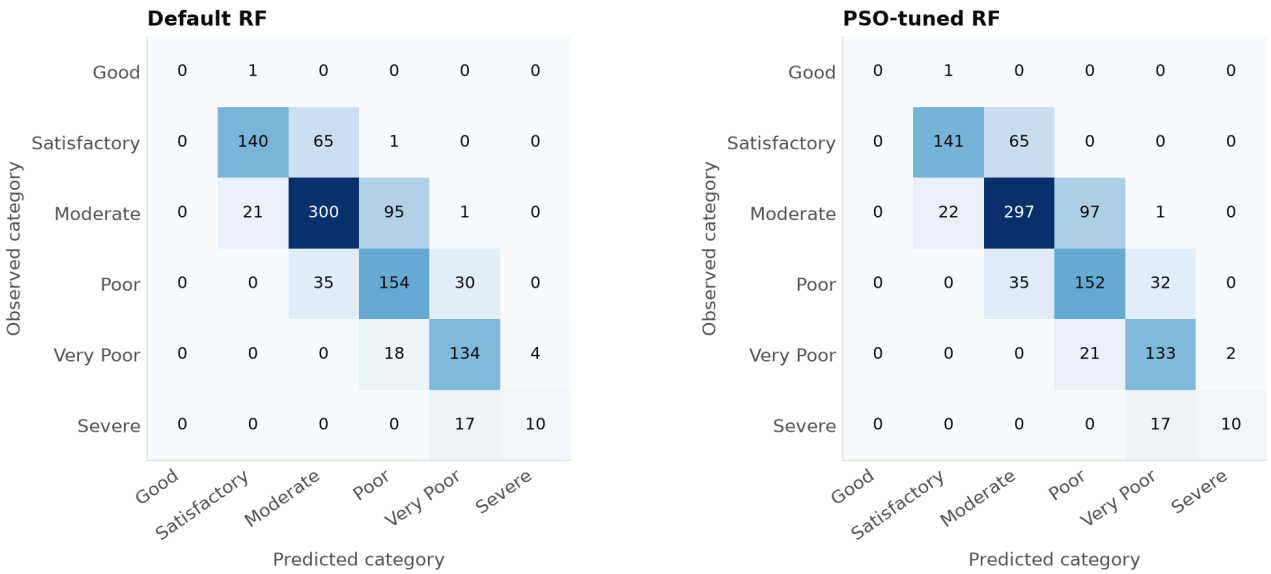


Figure 13: CPCB category confusion matrices on the test period

0.7.10 Computational Performance

Training Efficiency

Table 15: Computational resource usage (per run)

Metric	PSO joint	RS joint	PSO hyp.	RS hyp.
Fitness evaluations	312	312	312	312
Unique configurations trained	312	312	307–312	312
Forests trained per evaluation	5	5	5	5
Time per run (min)	14.2–24.8	11.8–12.4	24.3–46.6	20.7–22.7
Mean time per run (min)	19.5	12.1	35.2	21.8

The joint search is faster than the hyperparameter-only search, because each forest trains on fewer features. PSO runs take longer than random search because the swarm moves towards larger forests, which are slower to train. The search is embarrassingly parallel across runs.

0.7.11 Robustness Analysis

Stability Across Runs

Five independent joint PSO runs with different random seeds:

Table 16: Stability across independent joint PSO runs

Run (seed)	Val. RMSE	Test RMSE	Test R^2	Features	Trees / depth / leaf
0 (selected)	**39.73**	33.68	0.882	14	224 / 18 / 2
1	40.42	33.11	0.886	17	99 / 16 / 5
2	39.97	33.53	0.884	12	266 / 18 / 5
3	39.79	33.57	0.883	18	293 / 15 / 3
4	39.81	33.47	0.884	18	187 / 22 / 3
Mean	**39.94**	**33.47**	**0.884**	16	
Std Dev	**0.28**	**0.22**	**0.002**		

Test RMSE is averaged over 10 forest seeds (the default forest averaged 33.81, SD 0.15, over the same seeds). Every joint PSO run beat the default forest on both validation and test error. However, the ranking by validation error does not carry over to the test period: the selected run (best validation) is the worst of the five on test, and run 1 (worst validation) is the best on test.

Validation Overfitting in the Larger Search

This is the most important finding of the extension. With 38 dimensions and 312 evaluations, PSO can find feature subsets that fit the particular validation folds of 2015 to 2023 very well. Part of the 0.93-point validation gain from feature selection is therefore specific to those folds and does not transfer to 2024 to 2026. The hyperparameter-only search, with only 6 dimensions, has less freedom to overfit its validation folds: the standard deviation of its five test RMSEs is only 0.04 points, against 0.22 for the joint search. Remedies include a longer or more recent validation window, penalizing the number of features in the fitness, or averaging the forests found by several runs.

Learning Curves

With a fixed validation window (the last 20% of the training period), the gap between training and validation RMSE at full size is 21.1 points for the default forest (15.1 vs 36.2), 11.2 points for the hyperparameter-only PSO forest (24.8 vs 36.0) and 9.2 points for the final model (27.3 vs 36.5). PSO's regularization more than halved the overfitting gap of individual trees.

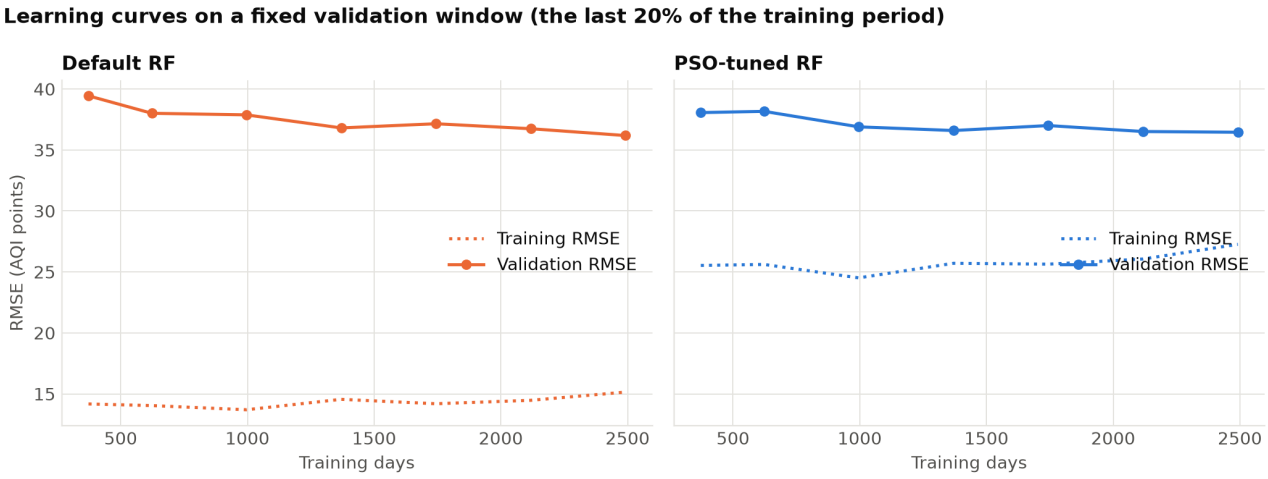


Figure 14: Learning curves of the default and final PSO Random Forest

0.7.12 Discussion

Why PSO Works Here

Several factors contribute:

1. **Guided search:** Sharing the global best moves particles quickly to good regions, which is why PSO beat random search in both search spaces, and by more in the larger one

2. **No gradient dependence:** The noisy, stepped validation error and the binary feature switches are handled naturally

3. **Exploration then exploitation:** Decreasing inertia gave a broad early search and a focused late search, visible in the diversity curve

4. **Physically sensible selection:** PSO kept wind, rain and radiation and discarded redundant humidity, pressure and lag features without being told which variables matter

Why the Test Gain over the Default Is Small

- Random Forest defaults are already strong on tabular data, so there is little error left for tuning to remove
- AQI history carries most of the predictive signal (88% of total importance), and every model uses it
- The largest gain came from features and model choice: moving from persistence to any learned model cut RMSE by 6 to 7 points; tuning changed it by 0.1 to 0.3
- In the joint search, part of the validation gain was overfitting to the validation folds

Limitations

- **Not statistically significant:** Neither PSO model is significantly better than the default forest on the test period
- **Validation overfitting:** The selected joint model was 2.3% better on validation than the hyperparameter-only model but 0.6% worse on test
- **Linear models competitive:** Ridge regression matched the forests, so the forest's complexity is not essential here
- **Reactive forecasts:** Sharp jumps are followed a day late
- **Observed, not forecast, weather:** The model uses today's weather; tomorrow's forecast weather could add information but is not in the archive used
- **Single city:** Results apply to Delhi

Practical Implications

- The model forecasts next-day AQI within about 25 AQI points on average, and flags *Very Poor* days with AUC 0.986
- It needs only 14 inputs instead of 32, which simplifies data collection for deployment
- Retraining each year is advisable, given the downward trend in Delhi's AQI

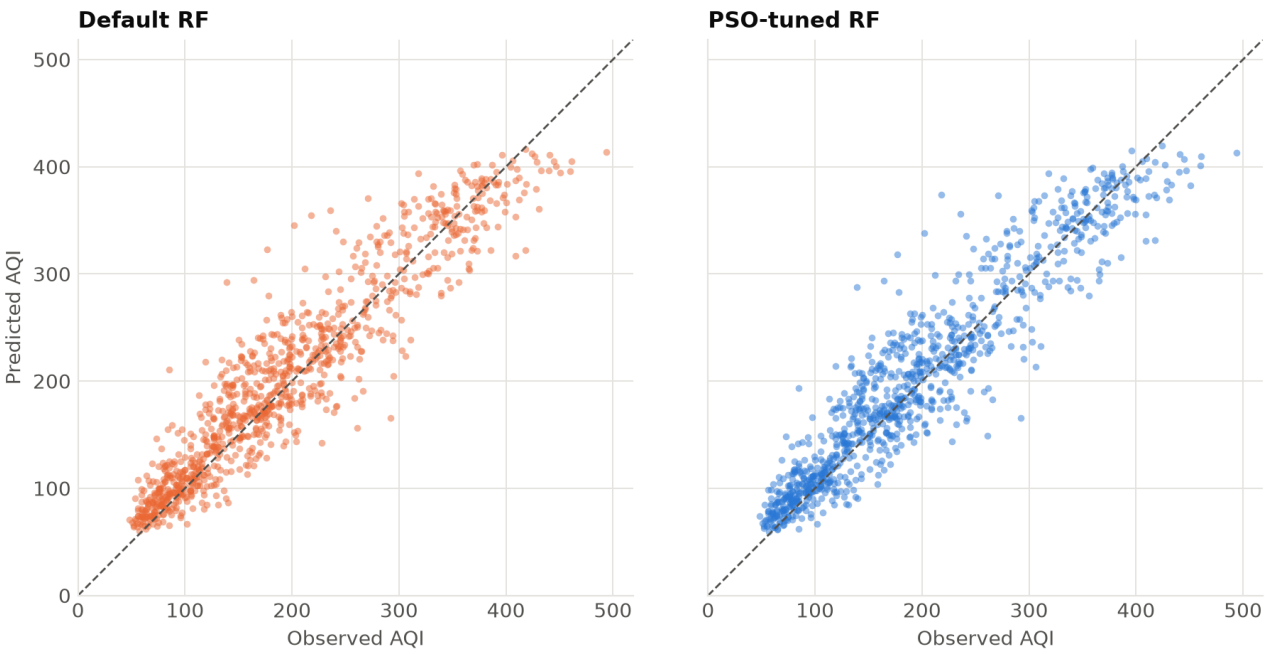


Figure 15: Model performance before optimization (left, default RF) and after PSO optimization (right)

0.8 Conclusion

This project built and evaluated a Random Forest optimized by Particle Swarm Optimization for next-day AQI forecasting in Delhi, using 4,170 days of official CPCB bulletins and ERA5 weather from 2015 to 2026. PSO was implemented from first principles and verified on benchmark functions, where it solved the Sphere function to machine precision.

PSO searched the six Random Forest hyperparameters and the choice of input features together, in a 38-dimensional space with threshold-based feature switches. It reduced walk-forward validation RMSE by 4.3% (41.52 to 39.73), twice the 2.1% reached by tuning hyperparameters alone, and it beat random search with the same budget in both search spaces, by a wider margin in the larger one. It kept 14 of 32 features, consistently choosing wind, rain and radiation and discarding redundant humidity, pressure and lag variables.

On the held-out 2024–2026 period, the selected model achieved an RMSE of 33.77 and an R^2 of 0.882, 16.0% better than persistence. Averaged over its five runs, joint PSO gave the lowest test RMSE of any search (33.47, 1.0% below the default forest). However, the selected model's gain over the default forest was small (0.25%) and not statistically significant, and it was slightly worse than the hyperparameter-only model, showing that the larger search partly overfitted its validation folds. PSO is an effective optimizer for this task, but for next-day AQI forecasting, good features and weather inputs matter more than tuning, and a larger search space must be paired with a more robust validation scheme.

0.9 Future Work

While this project establishes a solid foundation, several directions remain for future investigation:

0.9.1 Methodological Enhancements

Better Inputs

Forecast Weather: Use tomorrow's numerical weather forecast instead of today's observed weather, so the model can anticipate changes in wind and rain rather than react to them.

Fire and Emission Data: Add satellite fire counts from Punjab and Haryana and festival calendars, which drive the sharp autumn jumps the model currently misses.

Multi-City Data: Train one model across several Indo-Gangetic cities to learn shared regional patterns.

Improved Optimization Strategies

More Robust Fitness: Penalize the number of features, use a more recent validation window, or evaluate each particle on several forest seeds, so the joint search overfits its validation folds less.

Ensembles of Runs: Average the forests found by several PSO runs; the five-run mean test RMSE (33.47) suggests this would be more reliable than picking one run.

Other Nature-Inspired Optimizers: Compare PSO with Genetic Algorithms, Differential Evolution and Grey Wolf Optimizer on the same budget and fitness function.

Weighted Fitness: Optimize an error that weights *Very Poor* and *Severe* days more heavily, since those forecasts matter most for public health.

Surrogate-Assisted PSO: Use a cheap surrogate model of the fitness to reduce the cost of each search.

Model Extensions

Gradient Boosting and Deep Models: Apply the same PSO framework to gradient boosting and to sequence models such as LSTMs, which have more hyperparameters and may gain more from tuning.

Probabilistic Forecasts: Provide prediction intervals, so that alerts can be issued with a stated level of confidence.

References

[1] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE International Conference on Neural Networks (ICNN'95)*, vol. 4, pp. 1942–1948, 1995.

[2] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE International Conference on Evolutionary Computation*, pp. 69–73, 1998.

[3] M. Clerc and J. Kennedy, "The particle swarm: explosion, stability, and convergence in a multidimensional complex space," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 1, pp. 58–73, 2002.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] P. Probst, M. N. Wright and A.-L. Boulesteix, "Hyperparameters and tuning strategies for random forest," *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 3, e1301, 2019.

[6] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[7] J. Snoek, H. Larochelle and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems 25*, 2012.

[8] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. San Francisco: Holden-Day, 1970.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.

[11] M. Dorigo, V. Maniezzo and A. Colorni, "Ant system: optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.

[12] S. Kirkpatrick, C. D. Gelatt and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[13] R. Storn and K. Price, "Differential evolution: a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[14] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.

[15] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989.

[16] S. K. Guttikunda and B. R. Gurjar, "Role of meteorology in seasonality of air pollution in megacity Delhi, India," *Environmental Monitoring and Assessment*, vol. 184, no. 5, pp. 3199–3211, 2012.

[17] Central Pollution Control Board, *National Air Quality Index*, Control of Urban Pollution Series CUPS/82/2014-15. New Delhi: CPCB, 2014.

[18] H. Hersbach *et al.*, "The ERA5 global reanalysis," *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999–2049, 2020.

[19] UrbanEmissions.info, "AQI_bulletins: daily AQI bulletins from CPCB for Indian cities," GitHub repository, `https://github.com/urbanemissions-info/AQI_bulletins`, accessed 6 November 2026.

[20] Open-Meteo, "Historical Weather API," `https://open-meteo.com/en/docs/historical-weather-api`, accessed 6 November 2026.